Object Oriented Programming (CSEG1044)

Unit 02 – Lecture 05: Abstract Classes, ‘final‘, and Interfaces (Design Choices)

April 1, 2026

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Quick Links

Overview Key Topics Demo Summary

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Agenda

1 Overview

2 Key Topics

3 Demo / Example

4 Summary

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Classes, Inheritance, Packages, and Interfaces

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Classes, Inheritance, Packages, and Interfaces
- **Course thread:** Use Abstract Classes, ‘final‘, and Interfaces (Design Choices) to strengthen the domain model, APIs, and access rules inside Campus Resource Manager.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Recall Before You Start

- **Inheritance**: Abstract classes extend inheritance by allowing an incomplete but shared parent design. Main reference: `Unit 02 / Lecture 04 slides/notes`.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Recall Before You Start

- **Inheritance**: Abstract classes extend inheritance by allowing an incomplete but shared parent design. Main reference: `Unit 02 / Lecture 04 slides/notes`.
- **Interface idea**: You already saw interfaces as contracts; this lecture compares them with abstract classes in more depth. Main reference: `Unit 02 / Lecture 03 slides/notes`.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Learning Outcomes

- Use abstract classes for shared code + required methods.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Learning Outcomes

- Use abstract classes for shared code + required methods.
- Explain `final` for variables/methods/classes.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Learning Outcomes

- Use abstract classes for shared code + required methods.
- Explain `final` for variables/methods/classes.
- Use interfaces (including `Comparable`, `Runnable`) as contracts.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Key Terms to Know

- **Abstract class**: class that cannot be instantiated; may include abstract methods.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Key Terms to Know

- **Abstract class**: class that cannot be instantiated; may include abstract methods.
- **Abstract method**: declared without body; must be implemented by concrete subclasses.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Key Terms to Know

- **Abstract class**: class that cannot be instantiated; may include abstract methods.
- **Abstract method**: declared without body; must be implemented by concrete subclasses.
- **Interface**: contract specifying methods a class must provide.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Key Terms to Know

- **Abstract class**: class that cannot be instantiated; may include abstract methods.
- **Abstract method**: declared without body; must be implemented by concrete subclasses.
- **Interface**: contract specifying methods a class must provide.
- **Final**: "cannot change" (variable), "cannot override" (method), "cannot extend" (class).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Key Terms to Know

- **Abstract class**: class that cannot be instantiated; may include abstract methods.
- **Abstract method**: declared without body; must be implemented by concrete subclasses.
- **Interface**: contract specifying methods a class must provide.
- **Final**: "cannot change" (variable), "cannot override" (method), "cannot extend" (class).
- **Contract**: what is promised by a type (what callers can rely on).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Abstract classes and methods* → `final` *keyword (design intent)* → *Interfaces: built-in + user-defined*.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Abstract classes and methods* → `final` *keyword (design intent)* → *Interfaces: built-in + user-defined*.
3. Trace the worked example and connect each line back to one concept frame.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Abstract classes and methods* → `final` *keyword (design intent)* → *Interfaces: built-in + user-defined*.
3. Trace the worked example and connect each line back to one concept frame.
4. Attempt the mini exercises before moving to the recap and exit check.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Unit 02: Classes, Inheritance, Packages and Interfaces

- Lecture focus: Abstract Classes, ‘final‘, and Interfaces (Design Choices)

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Today: Roadmap

- Abstract classes/methods; partial implementations

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Today: Roadmap

- Abstract classes/methods; partial implementations
- ‘final‘ variables/methods/classes (design intent)

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Today: Roadmap

- Abstract classes/methods; partial implementations
- ‘final‘ variables/methods/classes (design intent)
- Interfaces: built-in examples (‘Comparable‘, ‘Runnable‘) + user-defined interfaces

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Abstract class (when to use)

- Use when you want to share code + force subclasses to implement some methods.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Abstract class (when to use)

- Use when you want to share code + force subclasses to implement some methods.
- Abstract class can have fields + concrete methods + abstract methods.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Abstract class (when to use)

- Use when you want to share code + force subclasses to implement some methods.
- Abstract class can have fields + concrete methods + abstract methods.
- Cannot instantiate: `new Shape()` if `Shape` is abstract.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

`final` keyword (meaning)

- **final variable**: assigned once (constant-like).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

`final` keyword (meaning)

- **final variable**: assigned once (constant-like).
- **final method**: cannot be overridden.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

`final` keyword (meaning)

- **final variable**: assigned once (constant-like).
- **final method**: cannot be overridden.
- **final class**: cannot be extended (e.g., `String`).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Interface (when to use)

- Use when you want a contract with multiple implementations.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Interface (when to use)

- Use when you want a contract with multiple implementations.
- Reduces coupling: code depends on interface, not concrete class.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Interface (when to use)

- Use when you want a contract with multiple implementations.
- Reduces coupling: code depends on interface, not concrete class.
- A class can implement multiple interfaces.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Built-in interfaces (examples)

- `Comparable<T>`: defines natural ordering (`compareTo`).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Built-in interfaces (examples)

- `Comparable<T>`: defines natural ordering (`compareTo`).
- `Runnable`: defines a task (`run`) to execute in a thread.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Checkpoint and Reflection

Reflection prompt

Where would *Abstract Classes, 'final', and Interfaces (Design Choices)* matter most in Campus Resource Manager, and which design choice would you justify using this lecture's ideas?

Quick self-check

Which part needs one more example before you move on: *Abstract classes and methods*, *`final` keyword (design intent)*, the worked example, or the recap?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Example: Comparable for sorting

```
import java.util.Arrays;

class Student implements Comparable<Student> {
    private final String rollNo;
    private final double cgpa;
    Student(String rollNo, double cgpa) { this.rollNo =
        rollNo; this.cgpa = cgpa; }
    @Override public int compareTo(Student o) {
        return Double.compare(o.cgpa, this.cgpa); //
            descending
    }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Demo usage

```
public class Demo {
    public static void main(String[] args) {
        Student[] a = {
            new Student("21CS1001", 8.2),
            new Student("21CS1002", 9.1)
        };
        Arrays.sort(a); // uses compareTo
        System.out.println(Arrays.toString(a));
    }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Can we instantiate an abstract class?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Can we instantiate an abstract class?
2. What does `final class` prevent?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Can we instantiate an abstract class?
2. What does `final class` prevent?
3. One sentence: why use interfaces?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Watch-outs

- Trying to instantiate an abstract class directly.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Watch-outs

- Trying to instantiate an abstract class directly.
- Marking too many classes or methods final too early.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Watch-outs

- Trying to instantiate an abstract class directly.
- Marking too many classes or methods final too early.
- Choosing an interface even when shared state or shared code is central.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Key Takeaways

- Abstract class: share code + require methods.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Key Takeaways

- Abstract class: share code + require methods.
- Interface: contract + low coupling + multiple implementations.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Key Takeaways

- Abstract class: share code + require methods.
- Interface: contract + low coupling + multiple implementations.
- `final` communicates "no change" / "no override" / "no extend".

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Checkpoint Questions

1. When do you prefer interface over abstract class?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Checkpoint Questions

1. When do you prefer interface over abstract class?
2. Give one use case for `final method`.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Checkpoint Questions

1. When do you prefer interface over abstract class?
2. Give one use case for `final method`.
3. What does `Comparable` enable?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.
- Apply the idea once inside Campus Resource Manager to make the concept stick.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

Next Unit Preview

Next: Unit 03 – Nested Classes, Exceptions, Multithreading, I/O Streams

- We will start with nested classes and exception handling.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).